

MNEMOSYNE

Plateforme de suivi de scolarité.

23/01/2026

BELQASMI Rayan
BOURG Solène
ISHAQ Faizan

LIN HUANG Christophe
MERIEM Wissam
RABOUDI Yacine

SOMMAIRE

1. Compréhension du sujet et analyse des besoins.
2. Choix algorithmiques, techniques et de conception.
3. Démonstration + Questions
4. Méthodologie et répartition du travail.
5. Bilan et prise de recul.

Compréhension du sujet et analyse des besoins.



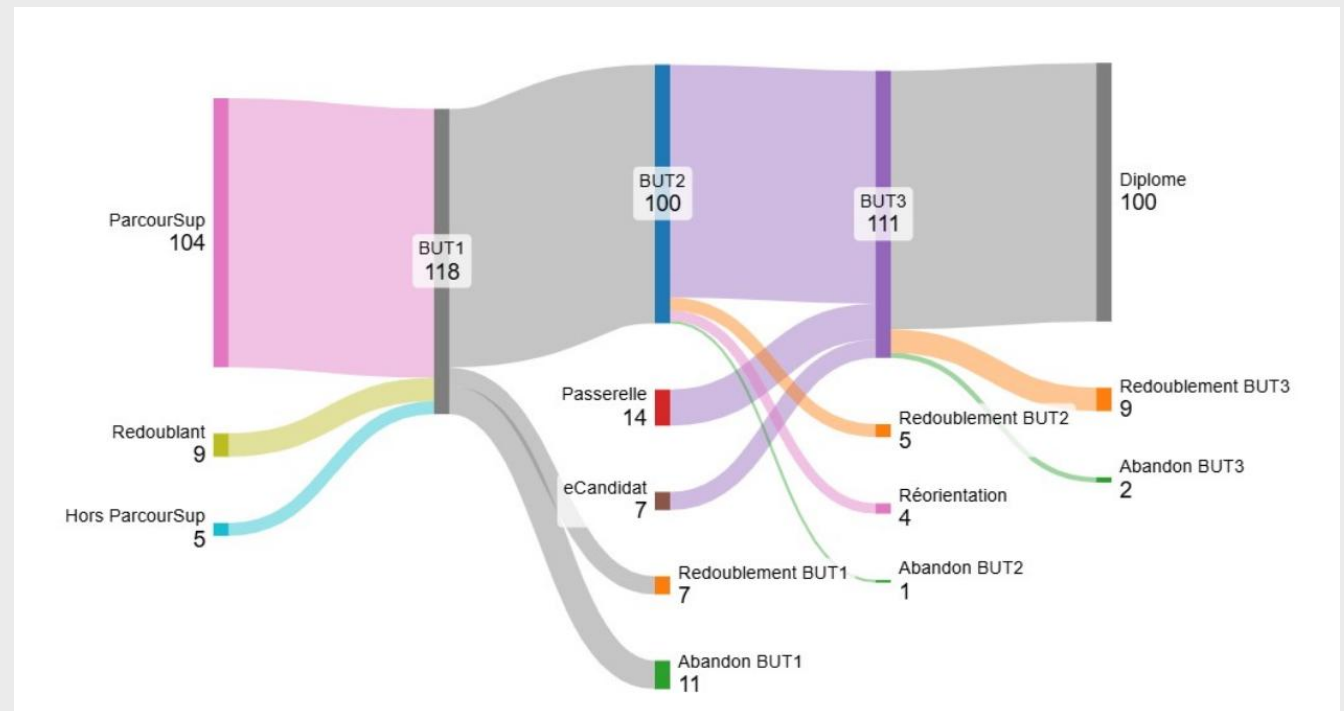
Compréhension du sujet et analyse des besoins.

Réforme 2021: BUT - GEA, GEII, INFO, SD, R&T, CJ



Compréhension du sujet et analyse des besoins.

Réforme 2021: BUT - GEA, GEII, INFO, SD, R&T, CJ



Exemple de cohorte

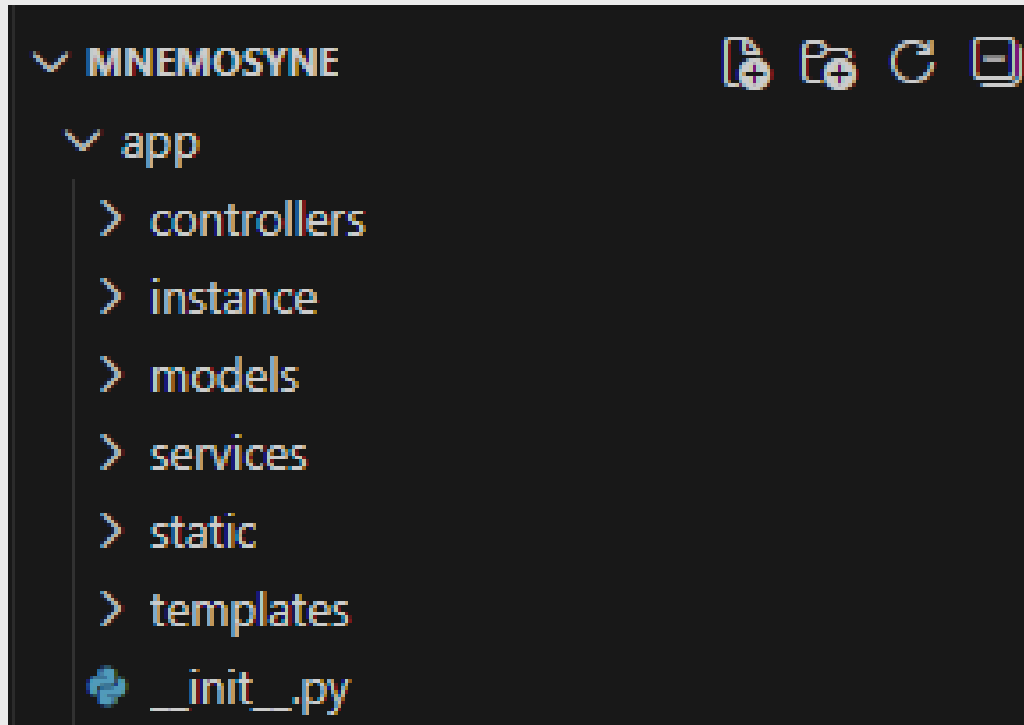
Compréhension du sujet et analyse des besoins.



Illustration de Mnémosyne réalisé par Zinaida Chernyshova

"Garder la mémoire, éclairer les parcours"

Choix algorithmiques, techniques et de conception.



Arborescence de notre projet

Architecture **MVC** (*Model View Controller*)

Choix algorithmiques, techniques et de conception.

```
@admin_bp.route('/admin', methods=['GET'])
@reqlogged
def admin_dashboard():
    """Affiche la page d'administration"""
    r = rs.get_regles()
    return render_template('admin.html', rules = r)
```

```
@admin_bp.route('/admin/delregle', methods=['POST'])
@reqlogged
def suppRegle():
    index = int(request.form.get('index'))
    rs.supprimer_regle(index)
    return redirect(url_for('admin.admin_dashboard'))
```

Méthodes dans AdminController.py

Architecture **MVC** (*Model View Controller*)

```
▼ MNEMOSYNE
  ▼ app
    ▼ controllers
      ├── __init__.py
      ├── AdminController.py
      ├── IndexController.py
      └── LoginController.py
```

Dossier "controllers"

Choix algorithmiques, techniques et de conception.

```
def _import_structure_formation(self, cursor, fmt, stats): ...  
  
def _create_passerelle_formation(self, cursor, dept_id, stats): ...  
  
def _import_resultats_semestre(self, cursor, fs_info, decisions_json, annee, c ...  
  
def _import_departements(self, cursor, depts_api, stats): ...  
  
def _import_referentiel_competence(self, cursor, scodoc_id, dept_id_bdd, map_c ...  
  
def _init_referentiels_statiques(self, cursor): ...
```

Méthodes dans ScoDocService.py

Architecture MVC (*Model View Controller*)

```
▼ MNEMOSYNE  
  ▼ app  
    > controllers  
    > instance  
    > models  
  ▼ services  
    📄 DonneeService.py  
    📄 RegleService.py  
    📄 ScoDocService.py  
    📄 ScoDocServiceV2.py  
    📄 SDS.py  
    📄 UserService.py
```

Dossier "services"

Choix algorithmiques, techniques et de conception.

```
def search_etudiants(self, annee_debut, dept, rythme, regles):  
    """Recherche dynamique pour le tableau de bord"""  
    db = self.get_db()  
    cursor = db.cursor()  
  
    params = [annee_debut]  
    sql_conditions = "WHERE i.annee_universitaire = ? + (f.annee_but - 1)"  
  
    if dept != "TOUS":  
        sql_conditions += " AND d.acronyme = ?"  
        params.append(dept)  
  
    if rythme != "TOUS":  
        if rythme == "FI":  
            sql_conditions += " AND f.id_rythme = 1"  
        elif rythme == "FA":  
            sql_conditions += " AND f.id_rythme = 2"
```

Méthodes dans DonneeDAO.py

Architecture MVC (*Model View Controller*)

```
▼ MNEMOSYNE  
  ▼ app  
    > controllers  
    > instance  
  ▼ models  
    DonneeDAO.py  
    Etudiant.py  
    RegleDAO.py  
    ScoDocAPI.py  
    User.py  
    UserDAO.py  
    UserDAOInterface.py
```

Dossier "models"

Choix algorithmiques, techniques et de conception.

ETL

1

Extract

2

Transform

3

Load

Choix algorithmiques, techniques et de conception.

ETL

1 Extract

```
def get_decisions_jury(self, formsemestre_id: int) -> List[Dict]:  
    """  
    Récupère TOUS les résultats (étudiants, décisions, moyennes) pour un semestre.  
    C'est ici qu'on trouve les notes et les validations de compétences.  
    """  
    return self._make_request(f"/formsemestre/{formsemestre_id}/decisions_jury")
```

Méthode "get_decisions_jury" qui récupère tous les résultat pour un semestre

Choix algorithmiques, techniques et de conception.

ETL

2 Transform

```
def _import_resultats_semestre(self, cursor, fs_info, decisions_json, annee, cache_etus
    """
    Importe les résultats en utilisant les champs précis 'modalite' et 'semestre_id'
    """

    # 1. Déterminer l'Année BUT (1, 2, 3)
    semestre_idx = fs_info.get('semestre_id')
    if not semestre_idx: return

    annee_but = math.ceil(semestre_idx / 2) # S1/S2->1, S3/S4->2, S5/S6->3

    # 2. Déterminer le Rythme (FI ou FA)
    # "FI" -> Formation Initiale
    # "FAP" -> Apprentissage
    modalite = fs_info.get('modalite', '').upper()

    # Si modalite contient 'FAP' ou 'APP', c'est de l'alternance (ID 2)
    id_rythme = 1 if 'FI' in modalite else 2
```

Méthode "import_resultats_semestre" qui importe les résultats

Choix algorithmiques, techniques et de conception.

ETL

3 Load

```
# Insertion dans la base
cursor.execute("""
    INSERT OR IGNORE INTO inscription
    (annee_universitaire, id_etudiant, id_etat, id_formation, id_decision)
    VALUES (?, ?, ?, ?, ?)
""", (annee, id_etudiant, id_etat, id_formation_bdd, id_decision))
```

Requête SQL avec cursor.execute

Choix algorithmiques, techniques et de conception.

```
# Map : ID ScoDoc (305) -> ID Dept BDD
scodoc_formation_ids = {}

for fmt in all_formations:
    # Filtre : On ne traite que les BUT
    titre = (fmt.get('titre') or fmt.get('titre_officiel') or '').upper()
    acronyme = (fmt.get('acronym') or '')

    # Condition plus large grâce à vos fichiers
    if 'BUT' in titre or 'BACH' in acronyme:

        # A. Création des 6 formations théoriques (BUT1/2/3 x FI/FA) pour ce département
        dept_id_bdd = self._import_structure_formation(cursor, fmt, stats)

        if dept_id_bdd:
            scodoc_id = fmt.get('id')
            scodoc_formation_ids[scodoc_id] = dept_id_bdd

# 4. Import des Référentiels (Parcours / Compétences)
# Map : (ID Dept BDD, Numéro UE) -> ID Competence BDD
map_competence_ids = {}

# On ne le fait qu'une fois par département pour éviter les requêtes inutiles
depts_traites = set()

for scodoc_id, dept_id_bdd in scodoc_formation_ids.items():
    if dept_id_bdd not in depts_traites:
        self._import_referentiel_competence(cursor, scodoc_id, dept_id_bdd, map_competence_ids, stats)
        depts_traites.add(dept_id_bdd)
```

Algorithme de déduplication

Choix algorithmiques, techniques et de conception.

```
# Map : ID ScoDoc (305) -> ID Dept BDD
scodoc_formation_ids = {}

for fmt in all_formations:
    # Filtre : On ne traite que les BUT
    titre = (fmt.get('titre') or fmt.get('titre_officiel') or '').upper()
    acronyme = (fmt.get('acronym') or '')

    # Condition plus large grâce à vos fichiers
    if 'BUT' in titre or 'BACH' in acronyme:

        # A. Création des 6 formations théoriques (BUT1/2/3 x FI/FA) pour ce département
        dept_id_bdd = self._import_structure_formation(cursor, fmt, stats)

        if dept_id_bdd:
            scodoc_id = fmt.get('id')
            scodoc_formation_ids[scodoc_id] = dept_id_bdd

# 4. Import des Référentiels (Parcours / Compétences)
# Map : (ID Dept BDD, Numéro UE) -> ID Competence BDD
map_competence_ids = {}

# On ne le fait qu'une fois par département pour éviter les requêtes inutiles
depts_traites = set()

for scodoc_id, dept_id_bdd in scodoc_formation_ids.items():
    if dept_id_bdd not in depts_traites:
        self._import_referentiel_competence(cursor, scodoc_id, dept_id_bdd, map_competence_ids, stats)
        depts_traites.add(dept_id_bdd)
```

Algorithme de déduplication

Choix algorithmiques, techniques et de conception.

```
# Map : ID ScoDoc (305) -> ID Dept BDD
scodoc_formation_ids = {}

for fmt in all_formations:
    # Filtre : On ne traite que les BUT
    titre = (fmt.get('titre') or fmt.get('titre_officiel') or '').upper()
    acronyme = (fmt.get('acronym') or '')

    # Condition plus large grâce à vos fichiers
    if 'BUT' in titre or 'BACH' in acronyme:

        # A. Création des 6 formations théoriques (BUT1/2/3 x FI/FA) pour ce département
        dept_id_bdd = self._import_structure_formation(cursor, fmt, stats)

        if dept_id_bdd:
            scodoc_id = fmt.get('id')
            scodoc_formation_ids[scodoc_id] = dept_id_bdd

# 4. Import des Référentiels (Parcours / Compétences)
# Map : (ID Dept BDD, Numéro UE) -> ID Competence BDD
map_competence_ids = {}

# On ne le fait qu'une fois par département pour éviter les requêtes inutiles
depts_traites = set()

for scodoc_id, dept_id_bdd in scodoc_formation_ids.items():
    if dept_id_bdd not in depts_traites:
        self._import_referentiel_competence(cursor, scodoc_id, dept_id_bdd, map_competence_ids, stats)
        depts_traites.add(dept_id_bdd)
```

Algorithme de déduplication

Choix algorithmiques, techniques et de conception.

```
# Map : ID ScoDoc (305) -> ID Dept BDD
scodoc_formation_ids = {}

for fmt in all_formations:
    # Filtre : On ne traite que les BUT
    titre = (fmt.get('titre') or fmt.get('titre_officiel') or '').upper()
    acronyme = (fmt.get('acronym') or '')

    # Condition plus large grâce à vos fichiers
    if 'BUT' in titre or 'BACH' in acronyme:

        # A. Création des 6 formations théoriques (BUT1/2/3 x FI/FA) pour ce département
        dept_id_bdd = self._import_structure_formation(cursor, fmt, stats)

        if dept_id_bdd:
            scodoc_id = fmt.get('id')
            scodoc_formation_ids[scodoc_id] = dept_id_bdd

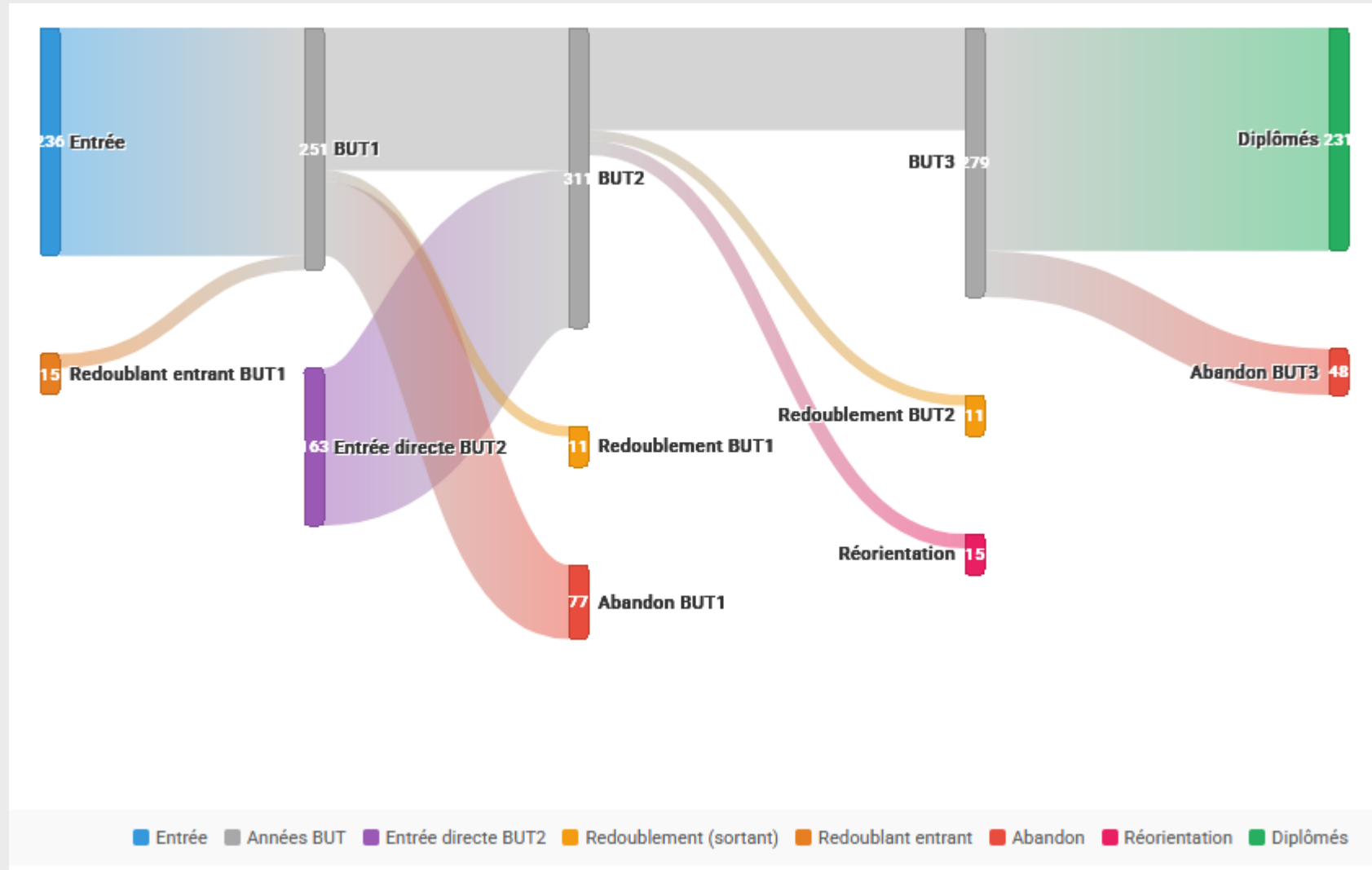
# 4. Import des Référentiels (Parcours / Compétences)
# Map : (ID Dept BDD, Numéro UE) -> ID Competence BDD
map_competence_ids = {}

# On ne le fait qu'une fois par département pour éviter les requêtes inutiles
depts_traites = set()

for scodoc_id, dept_id_bdd in scodoc_formation_ids.items():
    if dept_id_bdd not in depts_traites:
        self._import_referentiel_competence(cursor, scodoc_id, dept_id_bdd, map_competence_ids, stats)
        depts_traites.add(dept_id_bdd)
```

Algorithme de déduplication

Choix algorithmiques, techniques et de conception.



Cohorte sankey

Choix algorithmiques, techniques et de conception.

```
# Récupérer toutes les inscriptions de la cohorte sur 3 ans + année précédente pour les redoublants entrants
query = f"""
SELECT
    e.id_etudiant,
    e.ine,
    i.annee_universitaire,
    f.annee_but,
    dec.acronyme as resultat,
    d.acronyme as dept
FROM etudiant e
JOIN inscription i ON e.id_etudiant = i.id_etudiant
JOIN formation f ON i.id_formation = f.id_formation
JOIN departement d ON f.id_departement = d.id_departement
LEFT JOIN decision dec ON i.id_decision = dec.id_decision
WHERE i.annee_universitaire BETWEEN ? AND ?
{sql_dept}
{sql_rythme}
ORDER BY e.id_etudiant, i.annee_universitaire
```

Récupération de la cohorte avec requête SQL

Choix algorithmiques, techniques et de conception.

```
# Organiser les données par étudiant
etudiants = {}
for row in rows:
    etu_id = row['id_etudiant']
    if etu_id not in etudiants:
        etudiants[etu_id] = {}
    etudiants[etu_id][row['annee_universitaire']] = {
        'annee_but': row['annee_but'],
        'resultat': row['resultat']
    }
```

Reconstitution des trajectoires

Choix algorithmiques, techniques et de conception.

```
annee0, annee1, annee2, annee3 = annee_int - 1, annee_int, annee_int + 1, annee_int + 2
```

```
for etu_id, parcours in etudiants.items():  
    # Analyser le parcours de chaque étudiant
```

```
    # Redoublants entrants: étudiants qui étaient en BUT X l'année précédente et reviennent en BUT X cette année  
    if annee0 in parcours and annee1 in parcours:  
        if parcours[annee0]['annee_but'] == 1 and parcours[annee1]['annee_but'] == 1:  
            stats['redoublants_entrant_but1'] += 1  
        if parcours[annee0]['annee_but'] == 2 and parcours[annee1]['annee_but'] == 2:  
            stats['redoublants_entrant_but2'] += 1  
        if parcours[annee0]['annee_but'] == 3 and parcours[annee1]['annee_but'] == 3:  
            stats['redoublants_entrant_but3'] += 1
```

```
    # Année 1 (BUT1)
```

```
    if annee1 in parcours and parcours[annee1]['annee_but'] == 1:  
        stats['but1_total'] += 1  
        res = parcours[annee1]['resultat']  
  
        # Vérifier passage en BUT2  
        if annee2 in parcours and parcours[annee2]['annee_but'] == 2:  
            stats['but1_admis'] += 1  
        elif annee2 in parcours and parcours[annee2]['annee_but'] == 1:  
            stats['but1_redouble'] += 1  
        elif res and ('ADM' in res):  
            stats['but1_admis'] += 1 # Admis mais pas réinscrit  
        else:  
            stats['but1_abandon'] += 1
```

```
    # Entrées directes en BUT2 (nouveaux étudiants)
```

```
    if annee1 in parcours and parcours[annee1]['annee_but'] == 2:  
        stats['nouveaux_but2'] += 1
```

Classification dynamique

Choix algorithmiques, techniques et de conception.

```
annee0, annee1, annee2, annee3 = annee_int - 1, annee_int, annee_int + 1, annee_int + 2

for etu_id, parcours in etudiants.items():
    # Analyser le parcours de chaque étudiant

    # Redoublants entrants: étudiants qui étaient en BUT X l'année précédente et reviennent en BUT X cette année
    if annee0 in parcours and annee1 in parcours:
        if parcours[annee0]['annee_but'] == 1 and parcours[annee1]['annee_but'] == 1:
            stats['redoublants_entrant_but1'] += 1
        if parcours[annee0]['annee_but'] == 2 and parcours[annee1]['annee_but'] == 2:
            stats['redoublants_entrant_but2'] += 1
        if parcours[annee0]['annee_but'] == 3 and parcours[annee1]['annee_but'] == 3:
            stats['redoublants_entrant_but3'] += 1

    # Année 1 (BUT1)
    if annee1 in parcours and parcours[annee1]['annee_but'] == 1:
        stats['but1_total'] += 1
        res = parcours[annee1]['resultat']

        # Vérifier passage en BUT2
        if annee2 in parcours and parcours[annee2]['annee_but'] == 2:
            stats['but1_admis'] += 1
        elif annee2 in parcours and parcours[annee2]['annee_but'] == 1:
            stats['but1_redouble'] += 1
        elif res and ('ADM' in res):
            stats['but1_admis'] += 1 # Admis mais pas réinscrit
        else:
            stats['but1_abandon'] += 1

    # Entrées directes en BUT2 (nouveaux étudiants)
    if annee1 in parcours and parcours[annee1]['annee_but'] == 2:
        stats['nouveaux_but2'] += 1
```

Classification dynamique

Choix algorithmiques, techniques et de conception.

```
annee0, annee1, annee2, annee3 = annee_int - 1, annee_int, annee_int + 1, annee_int + 2

for etu_id, parcours in etudiants.items():
    # Analyser le parcours de chaque étudiant

    # Redoublants entrants: étudiants qui étaient en BUT X l'année précédente et reviennent en BUT X cette année
    if annee0 in parcours and annee1 in parcours:
        if parcours[annee0]['annee_but'] == 1 and parcours[annee1]['annee_but'] == 1:
            stats['redoublants_entrant_but1'] += 1
        if parcours[annee0]['annee_but'] == 2 and parcours[annee1]['annee_but'] == 2:
            stats['redoublants_entrant_but2'] += 1
        if parcours[annee0]['annee_but'] == 3 and parcours[annee1]['annee_but'] == 3:
            stats['redoublants_entrant_but3'] += 1

    # Année 1 (BUT1)
    if annee1 in parcours and parcours[annee1]['annee_but'] == 1:
        stats['but1_total'] += 1
        res = parcours[annee1]['resultat']

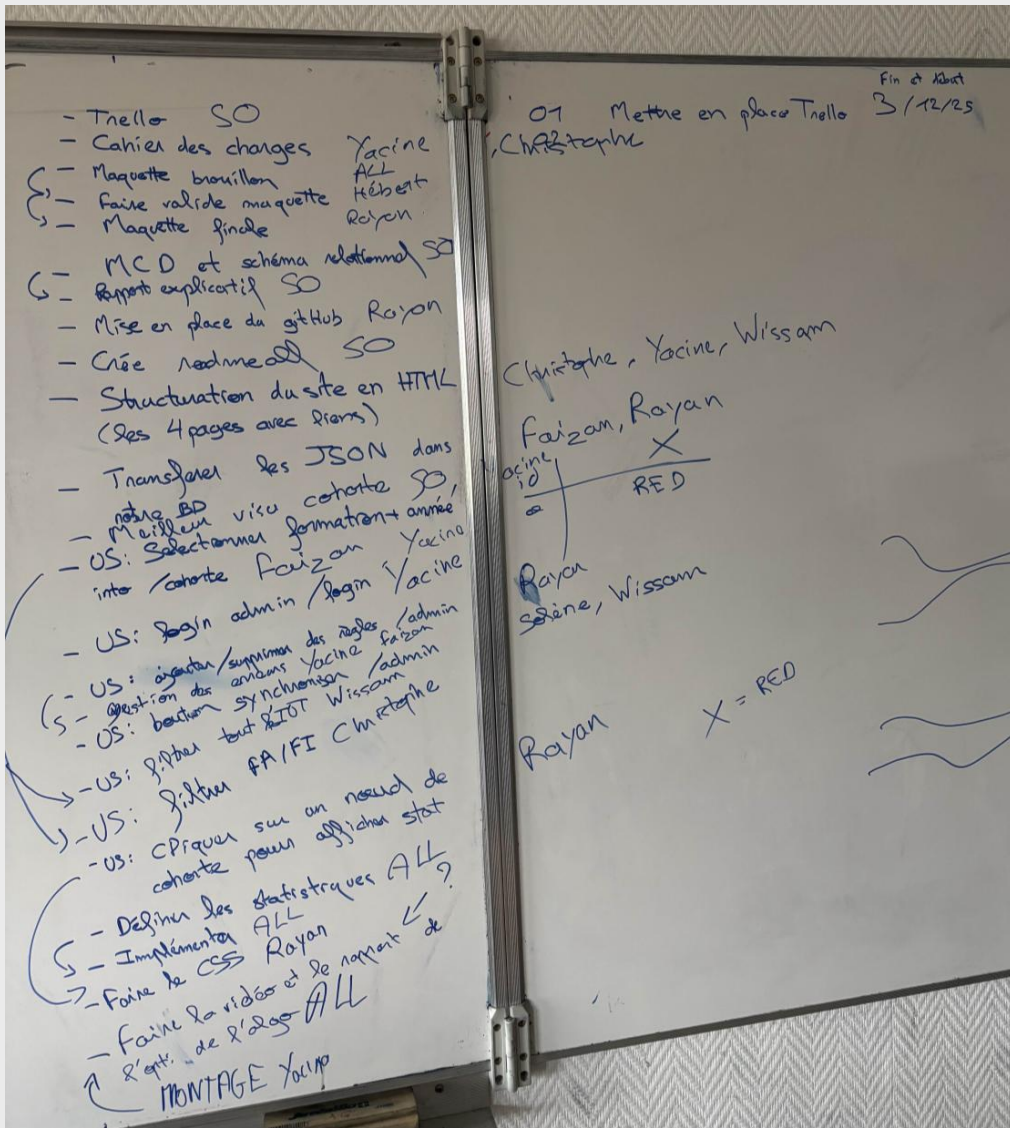
        # Vérifier passage en BUT2
        if annee2 in parcours and parcours[annee2]['annee_but'] == 2:
            stats['but1_admis'] += 1
        elif annee2 in parcours and parcours[annee2]['annee_but'] == 1:
            stats['but1_redouble'] += 1
        elif res and ('ADM' in res):
            stats['but1_admis'] += 1 # Admis mais pas réinscrit
        else:
            stats['but1_abandon'] += 1

    # Entrées directes en BUT2 (nouveaux étudiants)
    if annee1 in parcours and parcours[annee1]['annee_but'] == 2:
        stats['nouveaux_but2'] += 1
```

Classification dynamique

Démonstration

Méthodologie et répartition du travail.



3.2 User Stories

Nous avons structuré nos développements autour de besoins utilisateurs précis.

Profil : Client

- En tant que client je veux sélectionner une formation et une année universitaire afin de visualiser le suivi d'une cohorte d'étudiants spécifique sous forme de diagramme.
- En tant que client je veux pouvoir cliquer sur un nœud de la cohorte afin de voir les statistiques associées au nœud cliqué et afficher le diagramme associés à la cohorte du nœud cliqué.
- En tant que client je veux pouvoir filtrer la recherche par statut afin d'affiner les analyses.(par exemple : FI/FA/passerelle)
- En tant que client je veux pouvoir sélectionner toutes les formations afin de visualiser le suivi de toutes les cohortes de l'IUT et pas une seule formation.

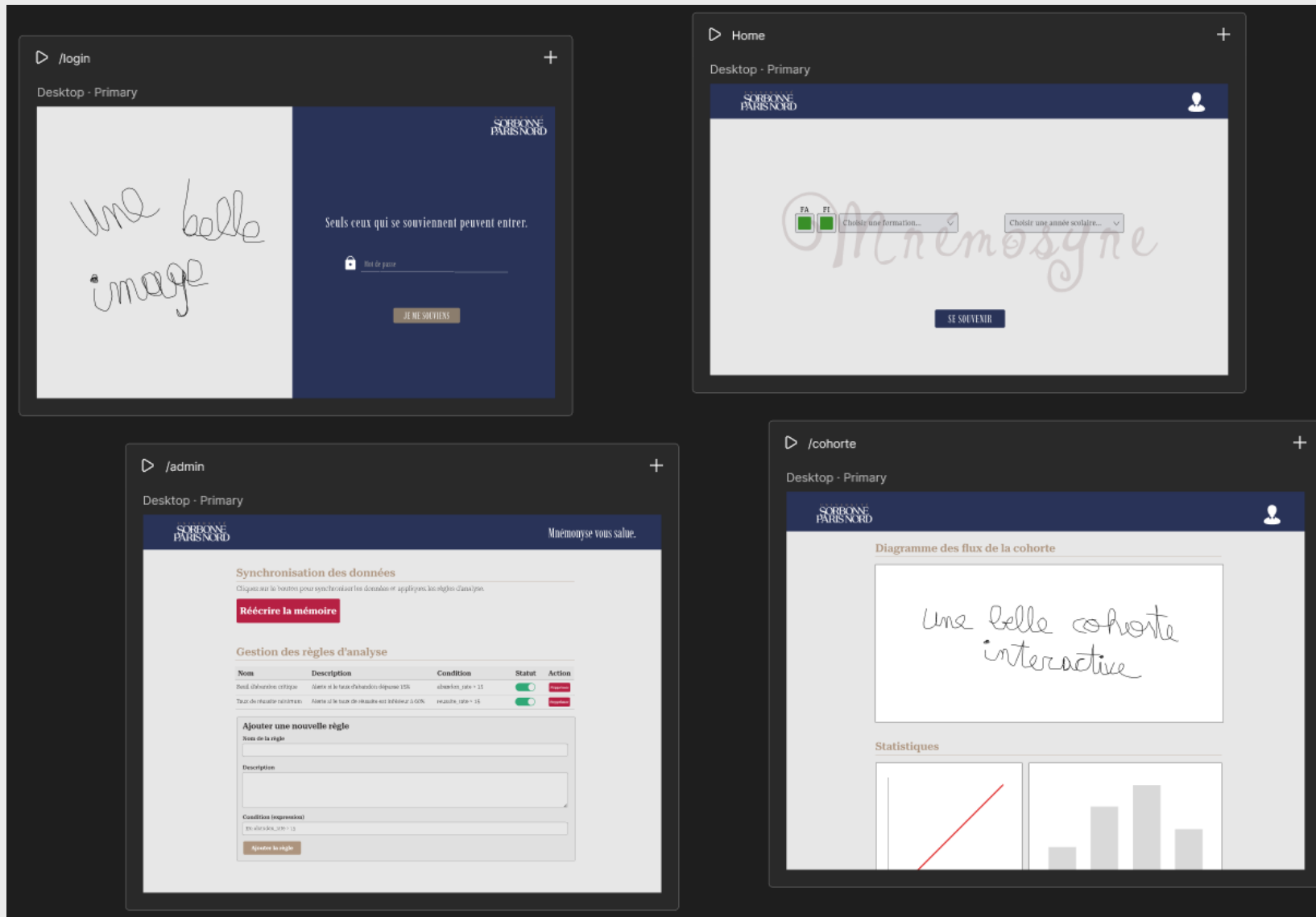
 Profil : Administrateur

- En tant qu'administrateur je veux pouvoir m'identifier afin d'accéder à la page administrateur.
- En tant qu'administrateur je veux pouvoir ajouter/supprimer une règle afin de changer les méthodes d'analyse des recherches.
- En tant qu'administrateur je veux pouvoir appuyer sur un bouton synchroniser afin que les données et les règles soient mises à jour.

User stories

Tableau brouillon de répartition des tâches

Méthodologie et répartition du travail.



Maquette Figma

Méthodologie et répartition du travail.



Tableau d'avancement + Feuille de temps

Méthodologie et répartition du travail.

<i>MEMBRES</i>	<i>TEMPS TRAVAILLÉ / PERSONNE</i>	<i>%</i>
BELQASMI Rayan	45,65	23,50%
BOURG Solène	42,15	21,70%
ISHAQ Faizan	33,05	17,01%
LIN HUANG Christophe	28,15	14,49%
MERIEW Wissam	19,8	10,19%
RABOUDI Yacine	25,45	13,10%
Total général	194,25	

Tableau de répartition du temps de travail

Bilan et prise de recul.

Points Forts	Limites et Contraintes
• Architecture MVC respectée & Code modulaire. • Interface sobre et intégrée (Charte graphique USPN). • Visualisation des données (Sankey) fonctionnelle.	• Tests en environnement simulé (JSONs vs API Réelle). • Sécurité de l'accès administrateur (Authentification simplifiée).

Bilan et prise de recul.



Logo GitHub

Bilan et prise de recul.



Logo GitHub



Logo PHP

MERCI

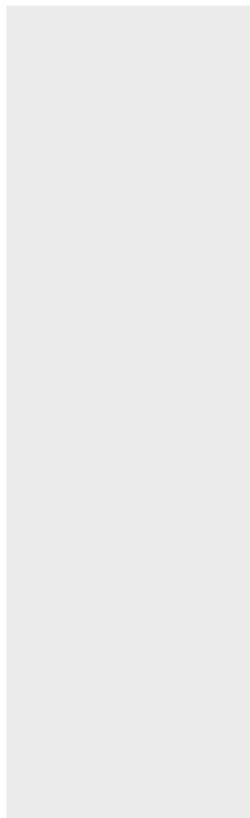
Principal



Secondaire



Fond



Accent



Titre : **JE SUIS UN TITRE**

Texte : Je suis un texte